

TECHNICAL REPORT · ARCHITECTURE REFERENCE

DIT Parking Manager System Architecture

How a layered Spring Boot application turns five services, a JPA class hierarchy, and one `ddl-auto=update` flag into a running parking system — and where each of the four OOP pillars actually shows up in the code.

Engineering Documentation

Version 1.0 · 2026.07

tz.ac.dit.parking

| Contents

01	Executive Summary	03
02	Service Layer	04
03	Database Schema Derivation	06
04	Domain Model & the Four OOP Pillars	08
05	Web Layer: DTOs, Controllers & Repositories	11
06	Appendix	13

| Executive Summary

DIT Parking Manager is a Spring Boot + PostgreSQL application with **zero hand-written SQL**: five services orchestrate all business rules, and the entire database schema is derived automatically from the Java domain classes. The codebase deliberately demonstrates all four object-oriented pillars – encapsulation, inheritance, polymorphism, and abstraction – as working mechanisms, not just vocabulary.

Key Takeaways

- Business logic lives in **5 services** orchestrating **6 repositories** over a domain model of **9 tables** – controllers stay thin, repositories stay dumb.
- The schema is generated by Hibernate at startup (`spring.jpa.hibernate.ddl-auto=update`): the one `JOINED` hierarchy (`Vehicle`) becomes a parent table plus one child table per subclass; every other entity is a single table.
- Fee calculation, payment processing, and authorization all resolve through a single polymorphic method call – **no if / switch** **type-checks** appear in the service layer for these decisions.

QUESTIONS THIS DOCUMENT ANSWERS

How do the five services divide responsibility, and which invariants does each one own? How does the database come to exist without a single line of SQL DDL? Where, specifically, do encapsulation, inheritance, polymorphism, and abstraction show up in this codebase?

| Service Layer

Services are the only layer allowed to talk to more than one repository. Each is annotated `@Service` and `@Transactional`, so every public method runs inside its own database transaction – if any step throws, everything in that method rolls back together.

Five services, five owners

Service	Owns	Key invariant enforced
<code>VehicleService</code>	Vehicle registration & visibility	Customers may only register vehicles for themselves
<code>SlotService</code>	Slot creation, resize, reservation	A slot cannot be resized while occupied
<code>ParkingService</code>	Check-in / check-out workflow	A vehicle cannot hold two unclosed sessions
<code>PaymentService</code>	Payment processing	Payment, session-close, and slot-release happen atomically
<code>ReportService</code>	Revenue & occupancy aggregates	Read-only – never mutates state

ParkingService – the workflow orchestrator

`checkIn()` composes calls to `VehicleService` (resolve the vehicle) and `SlotService` (choose or validate a slot), then applies its own guards – `requireCanOperate` and `requireNotAlreadyParked` – before persisting a `ParkingSession` and its `Ticket` together. Fee calculation itself is **not** in the service: `checkOut()` simply calls `session.close(now)`, and the entity computes its own fee. `requireCanOperate()` is exposed publicly so other services can reuse the same authorization rule rather than re-implementing it.

PaymentService – cross-service delegation

Rather than querying `ParkingSessionRepository` directly, `PaymentService` depends on `ParkingService` to fetch the session and re-verify authorization – a clean example of layering discipline: it delegates instead of duplicating logic it doesn't own.

```

Payment payment = build(request, session, session.getFee());
PaymentResult result = payment.process();           // ABSTRACTION: method is unaware
                                                    // whether this is cash, card, or
                                                    // mobile money

if (!result.successful()) {
    throw AppException.paymentFailed(result.message());
}
paymentRepository.save(payment);
session.markPaid();
session.getSlot().release();

```

These three side effects – save the payment, mark the session paid, release the slot – must happen together. Because the method is `@Transactional`, a failure partway through rolls back all three: you can never end up with a paid session and a slot still stuck as occupied.

VehicleService & SlotService – the same build pattern

`VehicleService` uses a `switch` on the request DTO's `type` field to construct the correct concrete subclass (`Car` / `Truck` / `Motorcycle`) before saving. This is the one place type-branching is unavoidable – turning a flat DTO into a polymorphic entity – after which every downstream call is polymorphic again. `SlotService` needs no such switch: a slot's `type` is just a column, copied straight from the request.

ReportService – read-only aggregation

Class-level `@Transactional(readOnly = true)` signals to Hibernate that no dirty-checking is needed. It is the only service that accepts and returns DTOs directly (`RevenueReport` , `OccupancyReport`) rather than domain entities, since its output is pure aggregate data with no identity to preserve.

| Database Schema Derivation

There is no Flyway, no Liquibase, no `schema.sql` in this project. The Java domain model **is** the schema – Hibernate reads the `@Entity` classes and generates matching PostgreSQL DDL automatically at startup.

```
spring.datasource.url=jdbc:postgresql://127.0.0.1:5433/parking
spring.jpa.hibernate.ddl-auto=update
```

`ddl-auto=update` is convenient for a coursework/demo project – every schema change is just a code change – but it is not production-safe: it never drops or renames columns cleanly, and there is no migration history to audit or roll back.

Rule 1 – one table per entity class

Simple entities map to one table each, named via `@Table(name = "...")`. `ParkingSession` -> `parking_session`, `Ticket` -> `ticket`. `@Id @GeneratedValue(strategy = IDENTITY)` becomes a Postgres identity primary key; `@Column` attributes (`nullable`, `unique`, `length`, `precision` / `scale`) become the matching column constraints.

Rule 2 – one hierarchy uses `JOINED` ; the rest use a type column

`Vehicle` is the only abstract root that declares `@Inheritance(strategy = InheritanceType.JOINED)`, because its subtypes carry genuinely different *behavior* (each prices and sizes itself) and different columns. This produces **one table per class**: loading a subtype means Hibernate joins parent and child on the shared `id`, and loading the abstract type reconstructs the correct concrete class from whichever child table has a matching row.

Users, slots and payments went the other way. Their variants differed only by a rule or two and a couple of optional fields, so each is **a single table with a discriminating column** – no joins, no empty child tables, and a variant can be changed with an `UPDATE` instead of a row migration.

Table	Strategy	Variants
vehicle	JOINED → car / truck / motorcycle	doors / axle_count / has_sidecar
users	role column	ADMIN (staff_number) / CUSTOMER (disability_permit)
parking_slot	type column	STANDARD / VIP (reserved_for_id) / DISABLED
payment	method column	CASH (amount_tendered) / CARD (card_last4) / MOBILE_MONEY (msisdn)

Rule 3 – relationships become foreign-key columns

Every @ManyToOne / @OneToOne field, via @JoinColumn, becomes a foreign key: `Vehicle.owner` -> `vehicle.owner_id`; `ParkingSession.vehicle` / `.slot` / `.operator` -> three FKs on `parking_session`; `Ticket.session` -> a **unique** FK on `ticket.session_id`, enforcing exactly one ticket per session at the database level, not just in application code.

Rule 4 – enums become VARCHAR

@Enumerated(EnumType.STRING) stores `Role`, `SlotType`, `SlotSize`, `SessionStatus`, `PaymentMethod`, `PaymentStatus`, and `MobileMoneyProvider` as their string names ("ACTIVE", "CLOSED") rather than ordinal integers – safe against reordering the enum later. The first three carry the variant that used to be a subclass, so this rule is what makes the flattened tables readable in raw SQL.

NET EFFECT

The schema is a direct projection of the class model, regenerated on every startup – there is no separate schema-design step. Where a subclass earns its keep in behavior, it earns a table too (`Vehicle`); where the only difference was a label and a nullable column, the hierarchy collapses into a type column. Every object reference remains a foreign key either way.

| Domain Model & the Four OOP Pillars

The `domain/` package holds JPA entities with real behavior, not anemic getters and setters. One class hierarchy carries the polymorphic part of the model — `Vehicle` → `Car/Motorcycle/Truck` — while `User`, `ParkingSlot` and `Payment` are single classes that switch on a `role`, `type` or `method` enum. The behavior lives in the same methods either way; only the dispatch mechanism differs.

Encapsulation

State changes are guarded by methods, never exposed via public setters. `Vehicle.setPlateNumber()` is the **only** way to write a plate — it trims, uppercases, and validates the Tanzanian format before storing. `ParkingSlot.available` has no setter at all: only `occupy()` / `release()` can flip it, and `occupy()` throws if the bay is already taken, which is what prevents double-booking. `User.role` is set by the `User.admin()` / `User.customer()` factories and never exposed, so flattening the hierarchy did not turn `role` into mutable state. `ParkingSession.close()` sets `exitTime`, computes `fee`, and advances `status` together in one guarded method — you can never end up with an exit time but no fee, or an invalid state transition.

Inheritance

Shared fields and behavior — `id`, plate validation, ownership — live in the abstract `Vehicle` parent; each concrete subclass adds only what makes it different: `Car.doors`, `Truck.axleCount`, `Motorcycle.sidecar`, plus its own `hourlyRate()` and `requiredSize()`. This is the same hierarchy described in Chapter 03's schema mapping — inheritance in Java and inheritance in the schema are one design decision, not two.

The counter-example is just as instructive. `User`, `ParkingSlot` and `Payment` were once hierarchies too, but their subclasses overrode one method and added one nullable field apiece — not enough behavior to pay for a join on every read. Inheritance is a tool for varying *behavior*; when the only thing varying is a label, an enum column says it more honestly.

Polymorphism

This is the pillar the codebase leans on hardest. Call sites invoke one method and let the object decide the behavior — by runtime type for vehicles, by enum for the rest — replacing what would otherwise be `if / switch` chains spread across the services:

```
// ParkingSession.calculateFee() – neither vehicle type
// nor slot type is ever checked; each object prices itself
return vehicle.hourlyRate()
    .multiply(slot.rateMultiplier())
    .multiply(BigDecimal.valueOf(billableHours()))
    .setScale(2, RoundingMode.HALF_UP);
```

- `PaymentService.pay()` calls `payment.process()` without ever knowing if it's cash, card, or mobile money – the `switch` on `method` is sealed inside the entity.
- `User.canOperate(vehicle)` : admins always return true, customers check ownership. Services call this uniformly and never read `role` themselves.
- `ParkingSlot.accepts(vehicle)` : each slot type has its own acceptance rule – VIP checks reservation, DISABLED checks the permit – but `SlotService.firstAvailableFor()` just filters `slot` → `slot.accepts(vehicle)` generically.

One expression, no `if`, no `switch`, no cast: the vehicle prices itself, the slot applies its own multiplier.

— comment on `ParkingSession.calculateFee()`

Abstraction

What matters is the contract a caller sees, not whether a subclass sits behind it. `Payment`'s own documentation states it plainly: the service holds a `Payment` and calls `process()` – it never learns whether money arrived as cash, card, or mobile money, and flattening the three subclasses into a `method` column did not change that one line of `PaymentService`. The same holds for `User` : callers ask `canOperate()` or `isAdmin()` and never branch on the role themselves. Repository interfaces abstract away all SQL/Hibernate mechanics behind method signatures, and DTOs (Chapter 05) abstract the wire format away from the persistence model entirely.

| Web Layer: DTOs, Controllers & Repositories

Request flow is strictly one-directional through the layers: a controller parses a DTO, calls a service with domain objects, and maps the domain result back into a response DTO.

Controllers never see a repository, and repositories never see a DTO – services are the only seam between the two.

DTOs – plain records at the boundary

All request/response shapes are immutable Java `record`s with no JPA annotations, decoupling the HTTP contract from the persistence model. Response DTOs use a static `from(entity)` factory to flatten an entity graph for the client – `SessionResponse.from()` pulls `plateNumber` and `slotCode` out of nested `Vehicle` / `ParkingSlot` objects rather than serializing them directly, avoiding lazy-loading pitfalls. `VehicleResponse.from()` is the one place polymorphism is deliberately set aside: it `instanceof`-checks the vehicle subtype because a DTO must be a flat, JSON-friendly shape, not a polymorphic object.

Controllers – thin and delegate-only

Each `@RestController` (`SessionController`, `VehicleController`, `SlotController`, `PaymentController`, `ReportController`, `AuthController`) follows the same three steps: resolve the current user via `AuthenticatedUserProvider`, delegate entirely to a service, and convert the returned entity to a response DTO. No business logic sits in a controller.

```
@PostMapping("/check-in")
public SessionResponse checkIn(@RequestBody CheckInRequest request) {
    return toResponse(parkingService.checkIn(currentUser.current(), request));
}
```

`GlobalExceptionHandler` centralizes error handling, converting every `AppException` into a structured `ApiError` JSON response – controllers don't write their own try/catch blocks.

Repositories – declarative, no implementation code

Every repository extends `JpaRepository<Entity, Long>`. Query methods are either derived from their name (`findByPlateNumber`, `findByAvailableTrueOrderByCodeAsc`, `findByIdAndRole`) or written as explicit JPQL (`ParkingSessionRepository.totalRevenueBetween` via `@Query`). Because `Vehicle` uses `JOINED` inheritance (Chapter 03), a single `VehicleRepository.findAll()` transparently returns a mixed list of `Car`, `Truck`, and `Motorcycle` instances – Hibernate reconstructs the correct concrete class from the joined tables automatically.

LAYER CONTRACT, IN ONE LINE

Controller <-> DTO, Service <-> Domain entity, Repository <-> Persistence. Nothing skips a layer.

| Appendix

A. Key source files

Concern	File
Fee calculation & session state	<code>domain/ParkingSession.java</code>
Payment abstraction & strategies	<code>domain/Payment.java</code> (method <code>column, process()</code>)
Check-in / check-out workflow	<code>service/ParkingService.java</code>
Payment orchestration	<code>service/PaymentService.java</code>
Schema-affecting config	<code>src/main/resources/application.properties</code>
Central error mapping	<code>web/GlobalExceptionHandler.java</code>
End-to-end behaviour tests	<code>src/test/java/tz/ac/dit/parking/ApiSmokeTest.java</code>

B. Verification

The behaviour described in this report is pinned by a smoke suite of **18 tests** (`./mvnw test`) that drives the real HTTP layer, security filters, services and JPA mappings – only the browser and the Postgres instance are substituted, the latter by in-memory H2 generating the same schema from the same `@Entity` classes.

The suite deliberately targets the rules that moved out of subclass overrides when the schema was flattened: a disabled bay checking the permit column, a VIP bay honouring its reservation FK, a non-VIP bay *refusing* to be reserved, the 1.5x VIP multiplier, and each payment method's approve and decline path. Those are the assertions that would catch a regression in the type-column dispatch, so they are the ones worth having.

C. Glossary

JOINED inheritance – a JPA mapping strategy where each class in a hierarchy gets its own table, joined to its parent on primary key, instead of one merged table.

ddl-auto=update – a Hibernate setting that generates and incrementally alters database tables from entity classes at startup, with no migration history.

DTO (Data Transfer Object) – an immutable record shaping HTTP request/response payloads, decoupled from JPA entities.

PaymentResult – a record returned by `Payment.process()` carrying success/failure and a reference or message, used identically regardless of payment method.

D. Scope note

This report reflects the state of the `tz.ac.dit.parking` codebase as documented during this review. It is descriptive, not prescriptive – recommendations (e.g. replacing `ddl-auto=update` with versioned migrations before production use) are flagged inline where relevant rather than collected separately.